

---

# HackPit

## Full Capability Assessment & Agreed Plan

**Date:** 26 July 2026

**Scope:** backend · frontend · pipeline · docker · docs · knowledge base

**Measured against:** OSCP · PNPT · eCPPT · HTB CPTS · CRTP · HTB/CTF · bug bounty · client pentests

### Contents

#### 0. Verdict

##### 1. Remaining gaps (deliberately deferred)

- 1.1 No true PTY — deliberate partial
- 1.2 The human-approval model — tiering still open

##### 2. Feature-by-feature audit

Companion (KB / search / attack paths)

Cockpit (execution)

AD graph + orchestration

Detection footprint

Code scan (SAST)

Frontend

##### 3. Knowledge base assessment

Findings

Recommended ingestion, in priority order

##### 4. Source-usage audit — "did I use my sources properly?"

- 4.1 The headline finding: the pipeline only reads \*.md
- 4.2 Consolidation is lossy for some sources

4.3 The biggest missed idea — PentestGPT's Pentest Task Tree — is now BUILT

4.4 The structural pattern

5. What is genuinely excellent — do not refactor

6. Security posture of HackPit itself

7. What the assessment examined

## **PART II — WHAT REMAINS**

Standing policy (unchanged, still governs the project)

Explicitly deferred (with the reason)

## **PART III — BUILD LOG (Phases 1–4, shipped 2026-07-26)**

Decisions taken (D1–D13)

Phase 1 — reach a real target

Phase 2 — make it remember

Phase 3 — make it fast to drive

Phase 4 — content & goal-specific

Verification

Status

# 0. Verdict

The **architecture is genuinely strong** — the safety engineering and the LLM-grounding discipline are better than most commercial tooling.

The original problem this assessment found was *reach*: nothing in HackPit was fake or stubbed, but the container it executed into contained seven programs, so it could not finish a real HTB box, touch a real AD domain, run a bug-bounty recon sweep, or complete an OSCP-style engagement. **That execution substrate has now been rebuilt (Phase 1), the loop reasons over structured state instead of stdout tails (Phase 2), the drive-speed gaps are closed (Phase 3), and the content/goal work is done (Phase 4).** See Part III.

What remains is a short list of *deliberately deferred* items — a true PTY, risk-tiered approval, a Windows execution target for CRTP, and a VPS for blind callbacks — each deferred for a stated reason, not left undone. See §1 and Part II's "Explicitly deferred".

## 1. Remaining gaps (deliberately deferred)

The blocker sections for everything the four phases fixed have been removed (see Part III). These two are deferred by decision, not left undone.

### 1.1 No true PTY — deliberate partial

`session.py` uses `docker exec -i`, without `-t`. Phase 3 added a **persistent line-session** to `:kali` (state now carries across commands — `cd`, environment and background jobs persist), which was the load-bearing half of the old "`:kali` is a command runner, not a shell" gap. A **true PTY** was deliberately *not* added: full-screen tooling (`vim`, `top`, an interactive `msfconsole`, a raw `evil-winrm` shell, `python -c 'pty.spawn'` upgrades) still needs a real terminal and `xterm.js`. This is a conscious trade — readable, logged transcripts over terminal-escape handling — not an oversight. Revisit only if a real engagement demands full-screen interaction.

### 1.2 The human-approval model — tiering still open

**Current state:** every command needs individual `approved=true`; anything the danger heuristic flags additionally needs `dangerous_ack`; `:kali` is human-driven (typing *is* the approval). Phase 3 added **one-keystroke approve** (`Enter` = approve · `S` = skip · `Esc` = stop; a dangerous command never fires on `Enter` alone) — the cheapest win, done.

**Still open — risk-tiered approval.** Keying the requirement to a deterministically computed risk class rather than to who is asking:

| Class                | Examples                                                                | Gate                                                                |
|----------------------|-------------------------------------------------------------------------|---------------------------------------------------------------------|
| 0 · Passive / OSINT  | subfinder, amass -passive, waybackurls, gau, whois, dig, crt.sh, shodan | <b>Auto-run in scope.</b> Logged, no prompt.                        |
| 1 · Active read-only | nmap, httpx, whatweb, nuclei (info/low), ffuf/gobuster at a rate limit  | <b>Batch-approve a plan</b> of N commands. Live feed + kill switch. |

| Class                            | Examples                                                                            | Gate                                   |
|----------------------------------|-------------------------------------------------------------------------------------|----------------------------------------|
| 2 · Active testing               | sqlmap, nuclei high/crit, dalfox, hydra, password spraying                          | Per-command (unchanged).               |
| 3 · Destructive / state-changing | AD writes, DCSync, password reset, exploits with payloads, reverse shells, msfvenom | Per-command + red confirm (unchanged). |

Most of the machinery already exists ( `allowlist.dangerous_command_heuristic()` is the Class-3 detector; `arsenal` categories + `detection.loudness` give a 0/1/2 split; `scope.py` bounds *where*). Missing: the classifier itself and a "run plan" object approved once. The prerequisite — fixing `_looks_like_host()` so the scope check is trustworthy when it becomes load-bearing — **is now done** (Phase 3, step 12). Deferred until bug bounty becomes the primary activity (a time-boxed autonomy window would layer on top, later). **Regardless of what else changes: never remove the gate for anything the danger heuristic flags.**

## 2. Feature-by-feature audit

### Companion (KB / search / attack paths)

| Component                                         | State                         | Notes                                                                                                                                                                                                                                                                         |
|---------------------------------------------------|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| KB load + serve                                   | <b>Solid</b>                  | 1,601 entries in memory, exclusions dropped at the door and re-filtered defensively at query time.                                                                                                                                                                            |
| Hybrid search ( <code>pipeline/search.py</code> ) | <b>Solid</b>                  | BM25 (stdlib) + <code>nomic-embed-text</code> cosine, weighted RRF (lex 1.0 / vec 0.5 — correct call for a KB full of exact identifiers), tier boost, whole-query title bonus. Degrades to lexical when Ollama is down instead of 500-ing.                                    |
| Attack-path composition                           | <b>Solid, genuinely novel</b> | Writeup-first mode + KB-first mode. Retrieval seeded per-phase so no phase starts empty. Step eligibility filtering ( <code>is_step_eligible</code> ) is unusually careful — excludes writeups, defensive-hardening guides, tool-install meta-docs, personal logs, grab-bags. |
| Grounding / validation                            | <b>Best-in-class</b>          | Cited <code>entry_id</code> s must resolve or the step downgrades to <code>ai_suggested</code> ; commands come from the KB entry, never the model; <code>target_adaptation</code> is dropped entirely if it names an FQDN not in the supplied facts.                          |
| <code>substitute_target</code>                    | <b>Excellent</b>              | CIDR sentinel to avoid double-rewrite, host-position gating so <code>.env.example</code> and <code>&lt;script&gt;</code> survive, refuses to rewrite what it can't rewrite confidently.                                                                                       |
| <code>foreign_refs</code>                         | <b>Excellent</b>              | Honesty marker — reports a foreign host/AD domain rather than guessing a replacement. Right call.                                                                                                                                                                             |
| Channel-2 context grounding                       | <b>Good</b>                   | Writeup/methodology content as reasoning background, strictly separated from the step pool, with a leak guard over model output.                                                                                                                                              |
| Engagement sessions                               | <b>Works</b>                  | SQLite, per-step checked + pasted results, debounced autosave.                                                                                                                                                                                                                |

| Component         | State                                   | Notes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-------------------|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Report generation | Very good, now multi-template (Phase 4) | Evidence built <b>programmatically</b> , not by the LLM — the model was observed mis-transcribing a port number. Exam/format templates: OSCP (per-host walkthrough + a proof.txt table spliced from state), CPTS (exec-summary + findings register), H1/Bugcrowd (impact-first + a CVSS 3.1 score <i>computed</i> , not asserted). proof.txt/local.txt are real per-host state (auto-captured when a command reads a flag file, or pasted). Collision-proof fences; lab vs REAL-TARGET labelled; detection footprint appended grounded-only; optional red-team OPSEC roll-up (D10). |
| Assistant chat    | Works                                   | Session-aware, KB-grounded, deterministic citation extraction (scans the reply for ids that resolve).                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Scripts arsenal   | Works                                   | 1,028 scripts deduped from the KB across 6 groups.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

## Cockpit (execution)

| Component                             | State                                  | Notes                                                                                                                                                                                                                                                                                |
|---------------------------------------|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Gated executor                        | Excellent design, now properly toolled | Four ordered gates, mode split, argv-only (never a shell), SSE streaming, run records persisted. The image it execs into now ships ~63 of the 73 catalogued tools (Phase 1).                                                                                                         |
| Isolation gate                        | Real                                   | <code>assert_isolation_proven()</code> structurally inspects every attached network for <code>internal: true</code> . Not a comment — an actual check.                                                                                                                               |
| Engagement mode                       | Correct                                | Explicit entry required, fail-closed on unknown/exited id, per-command approval re-checked even on the prevalidated path (belt-and-suspenders in <code>iter_run</code> ).                                                                                                            |
| Scope model ( <code>scope.py</code> ) | Good                                   | Hosts, <code>*.wildcards</code> , CIDRs, <code>!exclusions</code> , <code>*</code> opt-out. Fail-closed. No DNS at match time (avoids the "resolve an out-of-scope name to decide it's out of scope" leak). The <code>_looks_like_host()</code> false-positives are fixed (Phase 3). |
| Recon-driven expansion                | Good                                   | Mines run output for hosts, sorts by scope, in-scope join the live allowed set, out-of-scope surfaced read-only. Cannot widen the scope. Capped and never silently truncating.                                                                                                       |
| Orchestrator loop                     | Now state-grounded                     | Proposes one command, never executes. Feeds on the structured state model + live task tree (Phase 2) instead of stdout tails.                                                                                                                                                        |
| Engagement state model                | New (Phase 2)                          | hosts/services/endpoints/credentials/findings, upsert-only, fed by output parsers + loot-file ingest; drives the planner and a UI panel. Executes nothing (AST-asserted).                                                                                                            |
| <code>:kali</code>                    | Now a persistent shell (Phase 3)       | One long-lived <code>docker exec -i sh ; cd /env/background</code> jobs persist. Same containment (hardcoded open container, human-only, audited, no isolation gate). No PTY by design (§1.1).                                                                                       |
| Live sessions                         | Well-designed, now toolled             | Start is a gated command; stdin is human-only and source-scan locked.                                                                                                                                                                                                                |
| HTTP repeater                         | New (Phase 4)                          | Compose/send/replay/diff. Argv-only curl inside the hardcoded open box (no shell parses a request field; body on stdin), human-only + source-scan locked like <code>:kali</code> , scope-checked against a named engagement, every send run-recorded.                                |
| Pivot / tunnels                       | New (Phase 4)                          | chisel/ligolo-ng lifecycle (human-only start/stop) + pure route resolution and a <b>visible</b> proxychains rewrite applied <i>before</i> the approval screen. A tunnel's subnet enters scope only via an explicit, audited amendment; recon expansion still cannot widen.           |

## AD graph + orchestration

| Component                     | State                   | Notes                                                                                                                                                                                                                                         |
|-------------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BloodHound parser             | Genuinely good          | Handles v4/v5/CE, zip/dir/json/bytes/mapping, reconciles naming drift, synthesizes DCSync from <code>GetChanges</code> + <code>GetChangesAll</code> , emits coverage warnings for missing collection methods. Real work.                      |
| Path engine                   | Correct                 | BFS shortest path over abusable edges only, abuse-rank tie-break, k-shortest-ish alternatives.                                                                                                                                                |
| Technique catalog             | Good                    | 25 edge kinds, KB-grounded with catalog fallback, target-type specialization ( <code>GenericAll</code> on a group → <code>AddMember</code> ).                                                                                                 |
| Orchestrator                  | Excellent safety design | The model picks an <b>edge index</b> , never a command. Cannot invent a host, cannot author a command, cannot reach an edge outside the collection. A pick outside the list is refused rather than repaired.                                  |
| <code>advance</code> endpoint | Excellent               | Advancement requires a <code>run_id</code> that was <b>approved</b> and <b>exited 0</b> , verified server-side.                                                                                                                               |
| Execution tooling             | Now present             | impacket, certipy-ad, bloodyad, netexec, evil-winrm, bloodhound.py, kerbrute, responder, mitm6 are in the image (Phase 1). Live collection/execution against a real domain is still untested; the feature was built on synthetic sample data. |

## Detection footprint

**Read-only, honest, well-sourced.** Real SigmaHQ rule UUIDs, real ATT&CK ids (Enterprise v19.1), and a verifier script ( `pipeline/detection_sources.py --verify` ) that re-checks every id against live upstream. The "describes what blue sees, never how to be seen less" line is enforced in code. **Scale:** ~133 command specs, 19 distinct ATT&CK techniques — a good purple-team *flavour*, not a coverage tool.

**C RTP conflict — now reconciled (Phase 4, D10).** CRTP includes AMSI/logging evasion, which the panel used to refuse outright. It now has an **additive second channel**: the blue-team detection view is byte-for-byte unchanged and still passes the never-prescribe guard, and a separate, opt-in `opsec` channel adds the offensive half — what makes a command loud, the quieter tradecraft, and, mandatorily, *what still records it*. The OPSEC channel has its own guard (it may never advise disabling/clearing/tampering with a sensor); the LLM may extend it for uncatalogued commands, marked `ai_suggested`. See Part III, Phase 4.

## Code scan (SAST)

**Well-built and genuinely safe.** Static-only invariant asserted at every `_spawn()` and again by `test_codescan_safety.py`. Deliberately orthogonal — imports nothing from the engagement/executor/scope model. **Limits:** 19 bundled Semgrep rules, Python/JS/TS only; no Java/Go/PHP/Ruby/C#. Useful for open-source bug-bounty targets; thin as a general SAST.

## Frontend

16 routes, real API wiring throughout, SSE streaming, no mocked data layer ( `cockpitSample.ts` is the only sample content, explicitly labelled). New in Phases 1–3: the arsenal availability band, per-run time-budget + detach controls, the engagement-state/task-tree panel, the credential-vault "use" action, and the persistent `:kali` shell UI. **Gaps:** no global current-target/engagement context, no engagement export/import, no multi-target view, and the accepted 10-error `react-hooks` lint baseline (documented; `next build` passes exit 0).

### 3. Knowledge base assessment

1,601 entries · 1,395 with commands · median body 3,000 chars.

Source distribution:

| Source                 | Entries   |
|------------------------|-----------|
| hacktricks             | 715 (45%) |
| madstuff               | 260       |
| some-hacking-resources | 233       |
| writeups               | 59        |
| payloadsallthethings   | 49        |
| claudе-bug-bounty      | 46        |
| peh-notes              | 43        |
| htb-writeups           | 41        |
| claudе-red             | 39        |
| oscp-cpts-notes        | 36        |
| decepticon             | 33        |
| htb-academy            | 13        |
| hackpit-authored       | 12        |
| htb-box-pdfs           | 9         |
| galaxy-checklist       | 7         |
| htb-my-resources       | 5         |
| shodan-dorks           | 1         |

Tiers: 1,249 tier-3 · 241 tier-2 · 111 tier-1 (your own).

#### Findings

1. **PayloadsAllTheThings payload depth** — now recovered (Phase 4, D11/D12). PATT's 66 `.txt` payload lists + the shodan dork list are ingested at **payload-level granularity** (64 `payload-set` + 2 `dork-list` entries, `no_merge` -guarded so consolidation can never collapse them again), each a searchable entry over a full sidecar corpus mounted read-only at `/payloads` ; 56 `oscp_tools` files went to the Scripts Arsenal. KB 1,601 → 1,667. See Part III, Phase 4.
2. **"Grounded in your own notes" is mostly "grounded in HackTricks."** Search boosts tier-1 ( `search.py:58` ), but with 111 tier-1 entries that lever has little to pull on. **Growing tier-1 is the highest-leverage KB work still available** — still open.

3. **Empty categories** (still open): cloud 2 · mobile 2 · iot 2 · forensics 1 · ics 1 · phishing 1 · supply-chain 1. Cloud and API bug bounty have effectively no coverage.
4. **HTB Academy = 13 entries** (still open). The CPTS syllabus, for a cert on the target list, essentially not ingested.
5. **Missing as retrievable topics** (still open): API security (GraphQL/REST), OAuth/SSO flows, race conditions, business logic, HTTP request smuggling, prototype pollution, per-language deserialization, SSRF→cloud-metadata chains.
6. **No CVE/exploit index** (still open). "Find the public exploit for this version" is the OSCP core loop and nothing serves it.

Items 2–6 are the recommended *next* KB batches (PortSwigger Academy, HTB Academy proper, HackTricks Cloud, a CVE index); they were never in scope for the four phases and remain open for a future enrichment pass.

### Recommended ingestion, in priority order

1. **PortSwigger Web Security Academy** — the single best web-security source, and structured
2. **HTB Academy modules** properly (CPTS syllabus)
3. **HackTricks Cloud**
4. **PayloadsAllTheThings** re-ingested at **payload-level granularity** (see §4)
5. **An exploit-db / CVE mapping** keyed on service+version

## 4. Source-usage audit — "did I use my sources properly?"

Measured on disk against the built KB, not inferred.

### 4.1 The headline finding: the pipeline only reads \*.md

pipeline/ingest.py:229 → `sorted(source_path.rglob("*.md"))`; pipeline/ingest\_notes.py:599 likewise.

Every `.txt`, `.json`, `.py`, `.sh`, `.csv`, `.yaml` file in every source tree was invisible to the pipeline.

| Source folder          | Files | .md | KB entries | Non-md skipped | Of which is <i>real content</i>                                                                    |
|------------------------|-------|-----|------------|----------------|----------------------------------------------------------------------------------------------------|
| hackdic                | 807   | 806 | 715        | 1              | —                                                                                                  |
| madstuff               | 798   | 797 | 260        | 1              | —                                                                                                  |
| some hacking resources | 233   | 233 | 233        | 0              | —                                                                                                  |
| PayloadsAllTheThings   | 481   | 134 | 49         | 347            | 66 <code>.txt</code> payload lists + 28 <code>.py</code> scripts                                   |
| oscp-cpts-notes        | 288   | 85  | 36         | 203            | 174 PNG screenshots (+ git internals)                                                              |
| oscp_tools             | 99    | 1   | 0          | 98             | 28 <code>.exe</code> binaries, ~22 <code>.ps1</code> / <code>.sh</code> / <code>.py</code> scripts |
| HTB_academy            | 48    | 16  | 13         | 32             | 3 PNGs — effectively nothing                                                                       |
| shodan-dorks           | 32    | 1   | 1          | 31             | 1 <code>.txt</code> dork list                                                                      |



| Source folder                   | Files    | .md | KB entries | Non-md skipped | Of which is <i>real content</i> |
|---------------------------------|----------|-----|------------|----------------|---------------------------------|
| Hacking-Tools                   | 31       | 1   | 0          | 30             | nothing — git internals only    |
| PRACTICAL ETHICAL HACKING NOTES | 110      | 45  | 43         | 65             | images                          |
| more new resources (writeups)   | 118      | 72  | 100        | 46             | images                          |
| htb my resources                | 5        | 5   | 5          | 0              | —                               |
| htb boxes                       | 10 (pdf) | 0   | 9          | —              | PDF path handled separately     |
| PentestTools                    | 0        | 0   | 0          | —              | empty folder                    |

**Scale:** stripping git internals, images and compiled binaries, the genuinely lost knowledge was roughly **95 files** — chiefly **PATT's 66 .txt payload lists** (the pipeline ingested the *explanations* and discarded the *payloads* — backwards for bug bounty), the **shodan-dorks .txt**, and **~22 oscp\_tools scripts**. **Fixed (Phase 4, D11/D12):** an additive Route-B ingester ( `pipeline/ingest_corpora.py` ) folded all 66 payload lists + both dork lists into the KB as `payload-set` / `dork-list` entries with full sidecar corpora, and 56 `oscp_tools` files into the Scripts Arsenal — without rewriting a single existing entry (verified byte-identical) and idempotently. Two env traps hit and handled: `rglob` silently omitted 2 of 66 files (OneDrive dehydration → union with `git ls-files` ) and 22 files were AV-locked (→ git-blob recovery).

## 4.2 Consolidation is lossy for some sources

`madstuff` 797 md → 260 (33%). `oscp-cpts-notes` 85 md → 36 (42%). `PayloadsAllTheThings` 134 md → 49 (37%). Some is correct deduplication into HackTricks; a 3:1 collapse on your OSCP/CPTS notes is worth an eyeball — those are exactly the entries you'd want surviving as tier-1/tier-2.

## 4.3 The biggest missed idea — PentestGPT's Pentest Task Tree — is now BUILT

The assessment's single highest-value reasoning-layer recommendation was PentestGPT's task tree: a *persistent, hierarchical, status-tracked engagement state the model reads from and writes back to every turn*. **This shipped in Phase 2** ( `backend/state/tasks.py` ) — layered `1 / 1.1 / 1.1.1` tasks with `todo` | `done` | `n/a`, updated as results arrive via model-proposed operations that code validates. It is retained here as the rationale for what was built. (Implementation note: HackPit's variant has the model return *validated operations* rather than rewriting the whole tree, so one bad response cannot silently rewrite the plan.)

## 4.4 The structural pattern

The pipeline is excellent at *techniques* and lossy on *lists, workflows and payload corpora*. A checklist is a *sequence*, a dork list is a *set*, a payload collection is a *corpus* — all three were forced through a technique-shaped, markdown-only schema and collapsed. **Both fixes shipped (Phase 4):** the Route-B ingester accepts non-markdown inputs, and the second entry shapes ( `meta.kind = payload-set / dork-list`, `no_merge` -guarded) survive consolidation intact. The `checklist` shape is defined and available; no checklist source was in the D12 scope, so none were ingested yet.

## 5. What is genuinely excellent — do not refactor

---

- **The safety architecture is real.** Source-scan regression locks on `:kali` and session stdin, four ordered gates, clean lab/engagement mode split, `assert_isolation_proven()` as a structural runtime check. Better than most commercial tooling. (The Phase 1–3 work preserved every one of these invariants; the test suite grew with it.)
  - **The grounding discipline is the best idea in the project.** Grounded-vs- `ai_suggested` labelling; `entry_ids` must resolve or the step is downgraded; commands come from the KB, never the model; **evidence spliced programmatically into reports rather than written by the LLM**; `foreign_refs` as an honesty marker; the Channel-2 leak guard.
  - **The AD orchestrator picks an EDGE, not a command.** A safety design that also improves output quality. The Phase-2 task tree follows the same principle (validated ops, not free-form rewrites).
  - **advance** requires an approved, exit-0 run — evidence-based state transition, verified server-side.
  - **The consolidation pipeline** — alias-key + cosine match, structural merge, idempotent, provenance preserved.
  - **The detection footprint's framing and sourcing** — real Sigma UUIDs with a drift verifier.
  - **substitute\_target** — CIDR sentinel, host-position gating, refusal to rewrite what it can't rewrite confidently.
  - **The BloodHound parser** — v4/v5/CE reconciliation with honest coverage warnings.
- 

## 6. Security posture of HackPit itself

---

- **No authentication anywhere.** No `Depends`, no key, no session. CORS restricted to `localhost:3000`. Honestly documented in code.
  - On a laptop this is fine. **On a VPS attack box it is an unauthenticated RCE endpoint** — `POST /cockpit/kali` (and the new persistent-shell routes) run arbitrary `sh -c` in a container with full network reach to your host and LAN. Auth is a hard blocker before any non-localhost deployment.
  - Secrets handling is correct: `llm_config.json` gitignored, API key written to disk but never returned, collector preview redacts the password/hash. The engagement-state DB (which now holds **real captured credentials**, by decision) is gitignored like the rest.
  - The repo is code-only; KB, sources, sessions DB and `.env` are all gitignored. A prior full-history audit confirmed no secrets were ever committed.
  - **Known:** a self-scan flagged SSRF and CORS misconfiguration in HackPit's own backend (memory obs #1496). Worth revisiting before any exposure.
- 

## 7. What the assessment examined

---

**Read in full:** every non-test backend module (executor, allowlist, config, sandbox, engagement, scope, kali, session, runstore, router, models; `attack_path`, `main`, `orchestrator`, `llm`, `report`, `chat`, `context_channel`, `sessions`; all of `adgraph`, `arsenal`, `detection`, `codescan`), `pipeline/search.py`, `docker/Dockerfile.sandbox`, `docker-compose.yml`, the QA report, and the live KB.

**Structurally mapped:** the frontend (routes, API surface, component→endpoint wiring, demo/mock scan — none found beyond the labelled `cockpitSample.ts`); the test files; `pipeline/consolidate.py`.

Source trees inspected on disk: `hexstrike-ai` (151 `@mcp.tool` defs), `mantishack`, `Decepticon`, `PentestGPT` (task-tree prompt read directly), `claude-red`, `claude-bug-bounty`, `Galaxy-Bugbounty-Checklist`, `hackdic`, `htb boxes`, the writeup trees, `some hacking resources`. File counts and markdown ratios measured per tree.

Verified live (assessment): container tooling (37-tool probe), SYN-scan capability, nuclei templates, container capabilities, KB statistics, ingester file-type globs.

---

## PART II — WHAT REMAINS

*Decided with Zaid, 2026-07-26. All four phases and every decision that drove them are in Part III. The build is complete; this section is only the handful of things intentionally left for later.*

### Standing policy (unchanged, still governs the project)

---

These are not open work — they are the rules the project runs by, and building is finished, so they no longer need a decision table. Kept here as the active policy:

- **D2 — Windows + Docker Desktop; a VPS is added later, only for bug-bounty callbacks.** Docker Desktop is a real Linux VM (WSL2); the one thing it can't give is a publicly reachable listener for blind SSRF/XXE/RCE. The repeater (Phase 4) sends and reads the *direct* response; the VPS piece is still deferred until bounty work needs it.
- **D5 — Per-command approval stays.** One-keystroke approve shipped (Phase 3); risk-tiering is deferred (§1.2). Every new Phase-4 execution surface (repeater, tunnels) preserves this: nothing autonomous, human-only where it matters, and the tunnel rewrite is *visible before* approval.
- **D8 — HexStrike is a reference, never an execution backend.** Its tools bypass all four gates, the target-lock, the scope model and the audit trail. Rejected, not deferred.
- **D9 — Accept the CRTP Windows-execution gap; no Windows VM.** CRTP's PowerShell/.NET tooling can't run on a Linux container; HackPit plans and writes up CRTP work, the lab work happens elsewhere. Windows-only tools are kept and *marked*, never listed as runnable.

### Explicitly deferred (with the reason)

---

- **A true PTY** — deliberate partial (§1.1); readable logged transcripts over terminal-escape handling. Revisit only if a real engagement demands full-screen interaction.
- **Risk-tiered / batch approval and time-boxed autonomy** — until bug bounty is the primary activity (§1.2). The prerequisite ( `_looks_like_host()` ) is already fixed.
- **A VPS for blind out-of-band callbacks (D2)** — a NAT'd laptop has no public listener; deferred until bounty work needs it.
- **Kali VM as an execution target; HexStrike as an execution backend** — rejected, not deferred.
- **Next KB enrichment batches** — PortSwigger Academy, HTB Academy proper, HackTricks Cloud, a CVE/exploit index (§3, items 2–6). Recommended, never in the four-phase scope; a future pass.

- **Authentication before any non-localhost deployment** (§6) — no decision needed until a VPS enters the picture, but a hard blocker the moment it does.
  - Screenshot capture, recon diffing/monitoring, checklist-driven runs.
-

# PART III — BUILD LOG (Phases 1–4, shipped 2026-07-26)

Branch `sandbox-kali-image` , committed locally, **not pushed**. Full hermetic safety suite green throughout (29 test files); both Docker proofs 4/4 (lab still egress-less; engage fully open); browser-verified with Ollama ( `qwen3:8b` ).

## Decisions taken (D1–D13)

---

Every decision that drove the build. D1/D3/D4/D6/D7 were the Phase-1 build; D9 is a standing accepted gap (Part II); D2/D5/D8 are standing policy (Part II); D10/D11/D12 were Phase-4 work, now built; D13 is this document.

- **D1 — Fix the sandbox image; HackPit is the attack box.** A Kali VM's advantages (root, raw sockets, VPN, `/etc/hosts` ) are all reachable in Docker via capabilities + `/dev/net/tun` . *Built (Phase 1).*
- **D3 — Capabilities + root on the ENGAGE sandbox only.** Lab keeps `cap_drop: ALL` + non-root. *Built.*
- **D4 — All three sandboxes get the new toolset; only the lab's *network* isolation stays.** *Built.*
- **D6 — Catalog describes reality, not aspiration.** *Built.*
- **D7 — Startup reconciliation check.** *Built.*
- **D9 — Accept the CRTP gap; no Windows VM; stop listing Windows-only tools as available.** *Built (marked, not removed).*
- **D10 — Allow evasion/OPSEC content as an additive second channel, keeping the blue view.** *Built (Phase 4).*
- **D11 — Fix the KB gap via Route B (additive merge), not a rebuild.** *Built (Phase 4).*
- **D12 — Ingest PATT payloads + shodan dorks into the KB; `oscp_tools` into the Scripts Arsenal.** *Built (Phase 4).*
- **D13 — Plan lives in this document.** *Done.*

## Phase 1 — reach a real target

---

Commits `e4e8de5` , `80a18d5` , `2d0928f` .

- **New sandbox image** ( `docker/Dockerfile.sandbox` ) ON `kalilinux/kali-rolling` + `kali-linux-headless` , with SecLists, the Kali `wordlists` set (rockyou unzipped) and **~13,400 nuclei templates** baked in (the lab has no egress, so nothing can be fetched at runtime). Thematic tool layers for the specific tools the assessment found missing — `impacket`, `netexec`, `evil-winrm`, `bloodhound.py`, `certipy-ad`, `bloodyad`, `responder`, `mitm6`, `smbmap`, `enum4linux-ng`, `gobuster/nikto/feroxbuster/subfinder/httpx/amass`, `metasploit`, `exploitdb`, `hashcat/john`, `chisel/ligolo-ng/proxychains/socat/ncat`, `openvpn` — plus `katana/kerbrute/waybackurls/dalfox/gau/rustscan/jwt_tool` as pinned release binaries (no Kali package). **Result: ~63 of 73 catalogued tools present.** Kali's setcap'd `nmap` / `fping` are stripped at build so `no-new-privileges` doesn't refuse to exec them.
- **Privilege split** ( `docker-compose.yml` ) — the **engage** sandbox runs `user: root` with Docker's default capability set + `NET_ADMIN` and `/dev/net/tun` , so `-sS` / `-sU` / `-O` , `masscan`, `tcpdump`, `privileged-port` binds

(responder/ntlmrelayx) and VPN all work. **Lab and :kali are untouched** — uid 1000, cap\_drop: ALL , no devices. Verified: engage does SYN scans and binds :445; lab/ :kali refuse -ss .

- **Per-request timeouts** (default 180s, ceiling 3600s) on /cockpit/exec and :kali ; **background/detached jobs** returning a run\_id and a reconnectable replay stream ( /cockpit/runs/{id}/stream ), with a reaper. :kali 's old hardcoded 60s is gone.
- **Loot volume** — backend/data/engagements → /loot on the engage and :kali sandboxes (deliberately **not** the isolated lab); each engagement works in /loot/<id> via docker exec -w , so nmap -oA output survives docker compose down .
- **Startup reconciliation** ( GET /tools ) — probes the running sandbox with one command -v sweep over every catalogued name+alias, filters the planner's prompt to installed tools, and surfaces the gaps in the arsenal UI. Windows-only tools (rubeus/powerview/mimikatz/winpeas) are marked platform: windows , kept for planning/write-ups, and never proposed.
- **Arsenal catalog** reconciled against the shipped image; the orchestrator prompt no longer hardcodes gobuster / nikto .

## Phase 2 — make it remember

---

Commits c606f6d (backend), a8f40ae (UI).

- **backend/state/ package** — the structured engagement state the assessment identified as the deepest gap: hosts → services , endpoints , credentials , findings as dataclasses over a shared SQLite store where **every write is an upsert** (re-running a scan corrects, never duplicates). Output **parsers** (nmap XML, httpx/ffuf/nuclei JSON, secretsdump) as pure functions, and a post-run **ingest** that reads both a run's stdout and any file it wrote into its loot directory (that is how nmap -oA populates state). The package **executes nothing** — AST-asserted, because ingest runs automatically after every command.
- **Pentest Task Tree** ( state/tasks.py ) — PentestGPT's central idea (§4.3): layered 1 / 1.1 / 1.1.1 tasks with todo | done | n/a , persistent per session, updated as results arrive. The model returns **validated operations** ( add\_subtask / mark\_done / mark\_na / ... ) that code applies against the stored tree; one bad op is rejected on its own and the rest still apply.
- **Orchestrator grounding** — the proposer prompt now leads with the accumulated state + live task tree, with raw output demoted to a short tail. Measured: **460 chars of state vs 6,600 of stdout tails** for 12 runs of one nmap, and each fact appears once instead of once per run. A tail window forgets; state does not.
- **Credentials in the prompt: real values** (Zaid's explicit decision) so the planner writes fully-formed commands — one constant ( render.INCLUDE\_CREDENTIAL\_SECRETS ) with a test proving the flip works, so the choice stays reversible.
- **UI** — the CockpitState panel: counters, the task tree (with "seed from plan"), hosts+services grouped, endpoints with status colours, credentials masked with per-row reveal + validated/untested, findings by severity.
- **Bonus fix found by the browser pass** — the arsenal /arsenal response model was silently dropping platform / runs\_here , so every tool badged "windows only". Fixed + regression-tested.

## Phase 3 — make it fast to drive

---

Commit b316b74 .

- **Step 11 — one-keystroke approve.** The guided loop takes `Enter` = approve, `S` = skip, `Esc` = stop, hints shown on the buttons. A dangerous proposal never fires on `Enter` (the explicit danger confirm is still required); shortcuts are ignored while a text field is focused.
- **Step 12 — `_looks_like_host()` fixed.** Flipped from "assume host unless the last segment is a known file extension" to a **positive test**: a dotted token is a host only if its last label is a plausible alphabetic TLD and not a file extension. Stops the false rejections ( `directory-list-2.3-medium` , `Mozilla/5.0` , `-oA scan.1.2` , version strings) while still catching real hosts; a genuine off-target host is still refused. This was the prerequisite for any future auto-run tier.
- **Step 13 — persistent `:kali` shell.** One long-lived `docker exec -i sh` instead of a fresh exec per command; each command is delimited over the pipe with a per-command sentinel so output and exit code read back cleanly. `cd /env/background jobs persist` (verified in-browser: `cd /tmp → export F00=... → echo cwd=$(pwd) foo=$F00` returned `cwd=/tmp foo=...` across three separate API calls). Every `:kali` containment invariant intact. No PTY, by design (§1.1). (Caught the Windows `\n → \r\n` stdin-translation bug in the process — the Linux shell was receiving `pwd\r` ; fixed with bare-LF stdin.)
- **Step 14 — credential vault.** Captured credentials fill the `<user>` / `<password>` / `<ntlm-hash>` / `<domain>` placeholders the AD templates carry, one click instead of retyping a hash. The placeholder→field mapping lives server-side ( `state/credvault.py` ) so front and back can't drift; a password fills `<password>` not `<hash>` and vice-versa; only credential placeholders are touched. Verified in-browser: "use `svc_sql`" turned `nmap -u <user> -p <password> ...` into `nmap -u svc_sql -p S3cr3t! ...` .

## Phase 4 — content & goal-specific

Commits `90fe260` , `885d776` , `bf46695` , `8486c56` , `937c420` .

- **Item 1 — KB Route B additive ingest (D11/D12).** `pipeline/ingest_corpora.py` — a targeted ingester over only the missing files, run *additively* on the built KB (never through the real pipeline, which would revert downstream enrichment and rewrite a 15 MB gitignored, Defender-sensitive file). Two regression-locked guarantees: existing entries pass through as **bytes** (no key/escape drift on the 1,601 it never looked at), and every line it owns carries `meta.corpus_ingest` so a re-run is byte-identical. Shapes (§4.4): **64** `payload-set` + **2** `dork-list` entries, all `no_merge` so consolidation can't collapse a corpus into a technique page; each holds a capped excerpt while the full list is a **sidecar** under `data/kb/payloads/` , mounted read-only at `/payloads` in all three sandboxes so ffuf/wfuzz point straight at it. **56** `oscp_tools` files went to the Scripts Arsenal as file-backed rows (a path to copy, not 20,000 lines of PowerView), 38 Windows-only ones kept and marked `runs_here=false` (D9). Two env traps handled: rglob's OneDrive omission (→ union with `git ls-files` ) and AV-locked files (→ git-blob recovery). KB 1,601 → 1,667; arsenal 1,028 → 1,137. The Defender exclusion on `HackPit\data` was added first.
- **Item 2 — evasion/OPSEC as an additive second channel (D10).** The blue-team detection view is **byte-for-byte unchanged** and still passes `assert_describes_not_prescribes` (a test proves every blue field is identical with and without the offensive half attached, and that the default `footprint()` carries no `opsec` key — so tagging/reports/run-annotation are untouched). `detection/catalog.py` gains a curated `OPSEC` table (10 notes keyed by spec key: what makes it loud, the quieter tradecraft, the tradeoff, and — mandatory — `still_recorded` , the honesty marker). `resolver.py` gains an opt-in `include_opsec` channel with its own guard ( `assert_opsec_is_separate` ): a note that fabricates the honesty marker or advises disabling/clearing/tampering with a sensor is rejected. `report.py` gains an off-by-default red-team OPSEC roll-up. The panel renders it as a distinct amber section. LLM may extend to uncatalogued commands, `ai_suggested` .



- **Item 3 — HTTP repeater.** `cockpit/repeater.py` mirrors `:kali` containment (hardcoded open container, human-only + source-scan locked, audited) and adds a scope check `:kali` lacks. **Argv-only curl**, never a shell — method/headers/URL are discrete tokens, the body rides on stdin, so a header of `a; rm -rf /` is one literal `-H`. A human clicking Send is the approval (no per-send prompt); the orchestrator/agent have zero path to `send`. When the send names an active engagement the URL host is scope-checked (out-of-scope → 403, nothing sent). Frontend `/repeater` : compose/send, response view, per-session history with edit-to-resend and a line-level body diff. The VPS-for-callbacks piece stays its own decision (D2).
- **Item 4 — pivot/tunnel routing.** `cockpit/tunnels.py` — chisel/ligolo-ng lifecycle (start the listener in the engage sandbox, hand back the agent one-liner to paste on the compromised host; human-only start/stop, source-scan locked). `route_for` / `wrap_command` are **pure** — they compute a **visible** `proxchains -q` prefix (chisel/SOCKS) or leave the command unwrapped with a route note (ligolo/interface), applied *before* the approval screen so the human approves the exact argv (silent routing was rejected for exactly this reason). A tunnel's subnet enters scope only via `engagement.add_pivot_subnet` — the one deliberate, audited widening path; recon expansion still cannot widen. Frontend `/tunnels` : start form, live tunnels with the copy-ready one-liner + per-subnet "add to scope (by hand)", and a pure route preview.
- **Item 5 — exam-mode report templates + proof.txt as per-host state.** `state` `Host` gains `local_txt / proof_txt + ownership()` ; captured automatically when a command reads a flag file ( `proof.txt / root.txt` → `proof`; `local.txt / user.txt` → `local` — a stray 32-hex hash is never mistaken for a flag) or pasted via `POST /sessions/{id}/state/proof` ; the state panel badges foothold vs owned. `report.py` gains four templates — standard, **OSCP** (per-host walkthrough + a proof.txt table spliced from state at `{{PROOF_TABLE}}` , computed not model-written), **CPTS** (exec-summary + findings register), **H1/Bugcrowd** (impact-first + a **CVSS 3.1** base score computed by `cvss31_base` , matching the official calculator). Every template keeps the shared grounding rules + the `{{EVIDENCE}}` splice. Frontend: a template picker on the report screen.

## Verification

- **Hermetic safety suite** ( `sh backend/run_safety_tests.sh` ) — green after every phase, expanded across all four with `test_phase1_runtime` , `test_state` , `test_scope_hostcheck` , `test_credvault` , `test_corpora` , `test_detection / test_detection_safety` (OPSEC channel + blue-view-unchanged), `test_repeater` , `test_tunnels` , `test_report_templates` , and persistent-shell containment tests in `test_kali` . **29 test files**.
- **Docker proofs** — `isolation_proof.sh` 4/4 (lab still cannot reach internet or host), `engage_open_proof.sh` 4/4 (engage has full reach).
- **Browser** — every UI surface exercised against a live Ollama backend per the testing rule, including the Phase-4 surfaces: the payload-set arsenal rows, the OPSEC red-team channel, the repeater send/replay/diff, the tunnels route preview, and the exam report templates with the proof table.
- **Frontend** — `tsc` clean, lint at the documented baseline (10 errors + 1 warning, unchanged), `next build` exit 0 (routes include `/repeater` , `/tunnels` ).

## Status

Everything is local on branch `sandbox-kali-image` , **unpushed**, awaiting review. **All four phases are complete**. The only remaining work is the deliberately deferred list in Part II.